

When Abstraction Becomes Indirection:

What C and C++ cost the agent that maintains them

Vasili Gavrilov

23 August 2026

Abstract

Abstraction hides detail because a person can hold only so much at once. An agent has no such limit, and pays instead for what it must reconstruct. C and C++ share a machine model, so what separates them is that layer alone.

Over 1.21 million lines of public C and 430 thousand of public C++, C++ carries 60.9 functions per thousand code lines against C's 28.4, and its median function has cyclomatic complexity 1. The extra length of C is redundancy; its indirection count is half.

Source size is four measures that disagree, and compression is the one that survives a change of writing density: for one behaviour by one author, C carries 2.66 times the lexical tokens of the Java and 1.73 times the compressed bytes. Memory separates the same way, C and Rust peaking within 1% of each other on identical input and Python at 12.7 times.

A project's highest-complexity function is nearly always the code at its edge, in 24 of the 27 measured here that have one: an input parser in most, its outward twin in the rest. That number describes where a program meets its input, not how it was written.

An agent ran 60 hypothesis-blind maintenance runs against one program in five languages, judged by hidden tests. All passed, the source under maintenance was at most 2.7% of context, and a run cost 25,603 tokens a turn, spent re-sending the conversation rather than reading the code. At one-file scale the representation cannot be seen at all.

1 Question

A class hides a representation, an interface hides an implementation, a container hides memory, a framework hides control flow. Each is a remedy for a limit of human working memory. An agent is not under that limit and is under others: the context it carries, the retrieval it performs, and the relationships it resolves before acting. A hidden implementation is not read for free by an agent; it is a relationship to resolve, which abstraction adds to rather than removes. This paper keeps the three costs apart:

1. **Representation:** what is in the repository. Lines, tokens, identifiers, functions, call edges.
2. **Reconstruction:** the relationships an agent must resolve to establish the behaviour a task touches.
3. **Maintenance cost:** context tokens, wall time, turns, failures, hidden-test pass.

The question is whether the first predicts the third, or whether the second has to be modelled to predict it.

The axis is the fraction of a program's behaviour that is written down in the program. C sits at the low end, close to a portable assembler: a call goes where the source says it goes. C++ is the first step off it and the cleanest to measure, sharing a machine model, a compilation model, a performance envelope and a library lineage with C, so a C to C++ pair varies the human-facing layer and holds the rest fixed. A framework is the far end of the same axis: an annotation, a naming convention, a dependency container or a cluster manifest decides what runs, and the code acting on it is not in the repository in any form.

Ada, Rust, Java and Python are not on this axis. Each changes the runtime, the memory model and the library ecosystem at once, so a ratio against any of them measures a bundle. They are here as controls, calibrating the effect a language change produces when the change is not the layer under study. The rungs that are on the axis have to be written, and Section 10 specifies them.

The efficiency objection to abstraction is thirty years old and it was answered. In the 1990s the argument against C++ was machine cost: virtual dispatch, allocation, exceptions, code size. Compilers closed most of it and the zero-overhead principle became a design rule rather than a claim. Section 7 measures what that is worth now, on the one compiled language here that abstracts heavily: Rust peaks 1% above C on identical input and finishes in the same time. Where a managed runtime is involved the old objection survives, at 2.6 times for Java and 12.7 for Python, but for compiled abstraction it is spent. It was also never this paper’s objection. Machine cost is not what an agent pays.

The answer is not obvious in either direction. Hiding a representation adds a relationship to resolve. Hiding a lifetime removes bookkeeping that would otherwise be carried correctly through every path. RAI and the standard containers are where abstraction plausibly lowers reconstruction cost.

2 What is withdrawn

An earlier draft reported C at 292 physical lines and 1,863 lexical tokens against C++17 at 218 and 1,369 for the key and posting subsystem of `ais`, the C++ 26.5% smaller. No tokenizer, file list or commit was recorded, and no C++ implementation of that subsystem exists in any of the author’s repositories [1]; the only C++ in `ais` is the Flutter runner the toolchain generates. The C side at `84e3ad5` is `key.c`, `key.h`, `post.c`, `post.h`: 397 physical lines, 266 code, 84 comment, 1,882 lexical tokens, 521 identifier tokens, 10 functions. The earlier C figure is close to the two `.c` files alone at some earlier commit. The C++ figure has no source and is withdrawn. Nothing below depends on it.

3 Material

Every repository is public under <https://github.com/Anode1>. Table 1 lists what was measured.

Repository	Language	What it is
<code>ais</code>	C	associative index; the five-language bench in <code>tests/perf</code>
<code>cjitter</code>	C	four stochastic searches and a uniform control
<code>bpnn</code>	C	backpropagation network and a tabular predictor
<code>linearr</code>	C, Java	streaming least squares; <code>java/</code> is the fit, line for line
<code>iac</code>	C	inter-agent dispatcher
<code>graphcrawl</code>	C, JS	depth-bounded graph navigator over a sorted text file
<code>mincdp</code>	C, Java	Chrome DevTools Protocol client, one file per language
<code>ljms</code>	Java	a work queue that is one database table
<code>aisgedcom</code>	Java	GEDCOM genealogy toolkit, written 2009 and 2011
<code>aisconvert</code>	Java	personal genomics toolkit, written 2009
<code>System A</code>	Java	a commercial web application in production

Table 1: Repositories measured; the commit each was measured at is recorded in `measurements.json`. The years for `aisgedcom` and `aisconvert` are their file headers’: both were imported to git in 2015 and no Java file has changed since.

3.1 Three sets of equivalent programs

None varies the abstraction layer on its own; each changes the runtime and the library ecosystem with it. They fix the baseline, the fixtures and the noise floor.

bench. `ais/tests/perf` holds one program in five languages: a scan of a store counting lines containing a substring, and the two-pointer intersection of two sorted posting lists. Its `LANG_COMPARISON` note [4] checks that all five agree on a 1,000,000-record index (379,090 matching records, 21,923 common ids); this paper’s hidden fixtures (Section 8) add six more, including empty, identical and disjoint lists and an overlap at the last element, and all five pass. One boundary: the Python idiomatic scan path is

`data.count(pat)`, counting occurrences where the other four count lines, so on a line holding the substring twice Python differs. The fixtures stay where the five agree, at most one occurrence per line.

mincdp. A header-only C client (`cdp.h`, `demo.c`) and a Java client (`Cdp.java`, `Demo.java`) with the same command set, each with a demo driving the same page and printing the same assertions. Equivalence rests on that demo, which was not re-run here.

linearr. By their own headers `Linearr.java` is the twin of `main.c` with `process.c`, `Regress.java` of `regress.c` line for line, and `Csv.java` of `csv.c`. make java fits every example with both and diffs the coefficient tables. The `Cmain.c` carries a scoring mode the Java lacks, so this pair’s C/Java ratio is an upper bound.

3.2 Measurement procedure

- *physical*: newline count of the file. *code, comment*: cloc 1.98 [8].
- *lexical tokens*: the count from a lexer in `measure.py` over the comment-stripped source. Identifiers, numbers, string and character literals, multi-character operators and single punctuation are one token each. Python is tokenized by the standard library’s `tokenize`, with `NEWLINE`, `INDENT` and `DEDENT` counted (they are Python’s braces and semicolons) and `NL` and `COMMENT` dropped.
- *identifier tokens*: identifier tokens that are not keywords of the language. *distinct*: the number of different ones.
- *o200k*: tiktoken 0.14.0 [10], encoding `o200k_base`, over the raw file and over the comment-stripped file. Anthropic’s tokenizer is not public; `o200k` is a proxy, and `cl100k_base` is in the JSON output and differs from it by under 4% on every set.
- *functions, NLOC, CCN*: lizard 1.24.0 [9]: functions found, their non-blank non-comment lines, their cyclomatic complexity. lizard does not parse Ada.
- *nesting*: the deepest brace nesting; for Python the deepest indent; for Ada the deepest block (`begin`, `loop`, `if`, `case`, `record`, `select`).
- *call graph (C only)*: libclang 18.1.1 parses every `.c` with `-std=c99` and the project’s include directories. Nodes are functions defined in the set; edges are calls between them; the longest chain is the longest acyclic path after cutting back edges in a depth-first search. Zero parse errors on every project.
- *fn macros*: `#define` with a parameter list.

Exclusions: `tests.c` in each C project (a unit-test file), `graphcrawl/c/web.c` (414 KB generated from `web/` by `embed.sh`).

4 Source measurements

Set	Files	Phys.	Code	Cmt.	Lexical	Ident.	Distinct	o200k		Fns	NLOC		CCN	
								raw	strip		med	max	med	max
bench/C	1	31	31	0	863	278	69	774	774	6	3.5	8	4.5	11
bench/Ada	1	133	127	0	1,071	359	82	1,373	1,373	n/a	n/a	n/a	n/a	n/a
bench/Rust	1	73	66	3	609	172	53	739	668	4	16	21	5	9
bench/Java	1	66	56	5	682	195	51	824	709	4	12	19	7	11
bench/Python	1	25	25	0	496	140	41	478	478	2	5	8	3.5	6
mincdp/C	2	540	419	64	4,467	1,197	185	6,932	5,743	24	12	51	5.5	25
mincdp/Java	2	213	151	36	1,682	510	151	2,615	2,032	18	5.5	23	2	6
linearr/C	7	2,135	1,525	396	12,179	3,480	428	25,526	17,918	66	6	217	2	73
linearr/Java	3	737	483	201	4,692	1,373	171	9,017	5,666	15	7	217	3	74

Table 2: The equivalent sets. lizard does not parse Ada. Brace or indent nesting is 3, 7, 4, 4, 3 for the bench in file order, 4 and 5 for mincdp, 6 and 6 for linearr. The bench C packs several statements per line (31 physical lines, 863 tokens), so its per-function NLOC is not comparable with the others.

4.1 What the ratios say

C over the other language, same job:

Pair	Lexical	o200k raw	Code lines
bench C / Ada	0.81	0.56	0.24
bench C / Rust	1.42	1.05	0.47
bench C / Java	1.27	0.94	0.55
bench C / Python	1.74	1.62	1.24
mincdp C / Java	2.66	2.65	2.77
linearr C / Java (upper bound)	2.60	2.83	3.16

The author’s C is the larger representation of the same job by 1.3 to 2.7 against his Java, and larger than Rust and Python on the bench. Only Ada is larger than the C.

The two token measures diverge on the bench and agree on the two programs. The bench C packs 863 tokens into 31 physical lines, 28 per line, and o200k rewards dense text: it costs 0.94 of the Java and 1.05 of the Rust in o200k while carrying 1.27 and 1.42 times their lexical tokens. On `mincdp` and `linearr`, at ordinary density, the two ratios agree to within a hundredth and a quarter. An agent pays the tokenizer’s count, and that count tracks lexical tokens only at ordinary density. Line counts are the worst of the three: the bench C has a quarter of the Ada’s code lines and four fifths of its lexical tokens.

4.2 The C style, in numbers

The `ais` style guide [3] asks for small functions, explicit state, minimal preprocessing and shallow module relationships. Table 3 measures the seven C projects (20,055 code lines), the author’s Java (10,051), and 39,621 code lines of a commercial project, against those claims. Test packages are excluded throughout, as are three third-party files carried in the two genomics toolkits (Sun’s `SwingWorker`, twice, and `SourcePortal`’s `UCProperties`).

Project	Files	Code	Lexical	Fns	Function NLOC			CCN				Nest	Macros	Call graph				
					med	p90	max	med	p90	max	>10			Edges	Fan-out	med	max	Chain
ais/c	40	10,597	82,135	438	13	38	601	5	14	253	92	8	11	802		1	55	18
cjitter/c	4	603	5,650	28	9	34	99	3	18	38	6	5	1	47		1	7	9
bpnn/c	26	3,224	30,007	123	9	53	307	3	20	141	20	6	2	207		0	30	6
linearr/c	25	2,956	24,656	133	7	43	217	3	15	73	20	6	11	239		1	34	9
iac	16	865	8,314	41	14	43	56	6	21	33	9	7	0	99		1	15	6
graphcrawl/c	11	1,238	8,709	54	14	42	108	6	13	40	10	4	0	93		1	14	10
mincdp/c	3	572	5,762	27	15	38	71	6	19	25	6	4	0	45		1.5	13	7
ljms/src	6	561	3,891	41	10	23	37	3	6	9	0	6						
aisgedcom/src	65	4,620	33,308	236	7	36	211	2	7	41	15	9						
aisconvert/src	52	4,236	29,357	225	9	37	133	2	9	44	17	9						
linearr/java	3	483	4,692	15	7	70	217	3	43	74	4	6						
mincdp/java	2	151	1,682	18	5.5	13	23	2	5	6	0	5						
System A	121	39,621	334,615	1,485	12	51	1,249	4	14	329	244	10						

Table 3: The C corpus, the author’s Java, and System A (production sources, tests excluded). Fan-out is calls from a function to other functions of the project; chain is the longest acyclic call path. Blank cells: not measured for Java.

The median claim holds: median function 7 to 15 lines, median CCN 3 to 6, the median function calling one other function of the project or none. The tail claim does not. `main` in `ais/c/main.c` is 601 lines at CCN 253, `handle` in `serve.c` 423 and 169, `feed_import_from_map` 302 and 111, `fit_stream` in `bpnn` 307 and 141. In `ais` 92 of 438 functions exceed CCN 10, and the longest call chain is 18 functions.

Ports carry the C’s shape: `linearr/java` has its C’s maximum function line for line (217 lines, CCN 73 and 74). Java written without a C original does not. `aisgedcom` and `aisconvert`, 8,856

code lines and 461 methods from 2009 and 2011, hold the median at 7 and 9 lines and CCN 2 and cut the tail to 32 of 461 methods over CCN 10 (7%, against 21% in `ais`) with maxima of CCN 41 and 44. `ljms` has nothing above CCN 9.

Table 4 names the maximum function of every project. In all thirteen it is an input parser: of a command line, an HTTP request, a WebSocket frame, a file read record by record, a library’s arguments. The mechanism differs and the role does not. The one project whose maximum is a leaf, `mincdp/java` at CCN 6, has no tail to place. Section 6 tests this outside the author’s code, where it holds in thirteen of sixteen.

Project	Highest-complexity function	What it parses	NLOC	CCN
<code>ais/c</code>	<code>main</code>	command line	601	253
<code>cjitter/c</code>	<code>cjitter.compare_masked</code>	its caller’s parameters and method mask	99	38
<code>bpnn/c</code>	<code>fit_stream</code>	a file streamed record by record	307	141
<code>linearr/c</code>	<code>main</code>	command line	144	73
<code>iac</code>	<code>main</code>	command line	47	33
<code>graphcrawl/c</code>	<code>main</code>	command line	108	40
<code>mincdp/c</code>	<code>cdp_ws_read_msg</code>	a WebSocket frame	34	25
<code>ljms/src</code>	<code>Processor.start</code>	rows taken from the queue	37	9
<code>aisgedcom/src</code>	<code>ArgsParser.setArgs</code>	command line	120	41
<code>aisconvert/src</code>	<code>ArgsParser.setArgs</code>	command line	133	44
<code>linearr/java</code>	<code>Linearr.main</code>	command line	217	74
<code>mincdp/java</code>	<code>Demo.isPng</code>	none: a leaf, and the project has no tail	8	6
System A	<code>doGet</code>	an HTTP request	1,249	329

Table 4: The function of highest cyclomatic complexity in each project, and the input it reads. Twelve of the thirteen parse something supplied from outside them; the thirteenth has no function above CCN 10.

Its size varies with how much work it does before delegating. The two toolkits’ parser sets properties and returns, the work following a call later from a 57-line `Main`: CCN 41 and 44. `ais`’s `main` parses the options and then runs a second `switch` of 28 cases executing each command in place: CCN 253. System A’s `doGet` and `doPost` decode a request and serve it in one method: CCN 329 and 300. Same boundary, same author’s habits, a factor of eight. System A is otherwise the author’s C profile in another language: median method 12 lines at CCN 4, p90 51 and 14, against `ais`’s 13, 5, 38 and 14, with 244 of 1,485 methods over CCN 10 (16% against 21%).

The median is a property of how the code is written and is the same everywhere here. The maximum is a property of how much the input parser does before it delegates. An experiment on “explicit C” conditions on both, and the phrase names only the first.

5 System A: a case report

System A is a commercial web application in production. Its name, its domain and its business logic are withheld at the owner’s instruction. Its structure is reported, on the convention of a clinical case report: the subject is anonymous, every measurement that does not identify it is given in full, and the conclusion is bounded to what one case can carry. The author is one of its developers and has full repository access. The numbers come from the same `measure.py` as the rest of this paper, run against the private tree.

Production Java	121 files, 39,621 code lines, 1,485 methods
Tests	129 files, 47,709 code lines
Front end	46,390 lines of JavaScript, no framework, no build step
Relational schema	37 tables
Persistence layer	39 classes, 25,772 lines, 679 methods
SQL	622 statements, all written in the source, 509 prepared
Methods returning the response body directly	205
Entity, DTO, mapper and repository classes	0
Data-carrier classes	1, of 209 lines
Imports of Spring, Jackson, JPA, Hibernate or Map-Struct	0
Deployed runtime	8 jars, 3.3 MB, of which the JSON layer is 24 KB
Path from HTTP to SQL	3 artifacts
Steps on that path with no file to open	0

Table 5: System A, measured. Tests are excluded from the production figures throughout this paper.

A `ResultSet` becomes the response body in the method that ran the query. No object graph stands between the database and the client, and nothing decides at runtime that is not written down.

The case establishes a negative. The seven-artifact path of Table 13 is not required to deliver this class of behaviour, because a system delivering it without them is in production. A case report carries that and nothing more: it does not show the short path is cheaper, safer or more common, and one case supports no claim about frequency. The comparison that would be the framework rung of Section 10, which does not exist.

The low end of the axis is therefore available in Java at 39,621 production lines, which makes it an architectural choice rather than a property of C. Its function profile is the author's C profile in another language, and its maximum sits at the boundary like every other maximum here.

6 Other people's code

The same script, over sixteen public projects by other people: five C systems, six C++ libraries and five Rust projects, 1.76 million code lines. It tests two of the paper's claims outside the author's hands and fixes the feature rates the C++ conditions of Section 10 are built at. Rust is included because it compiles to the same machine model as C and abstracts as heavily as C++, which makes it the third point on a line that would otherwise have two.

Each project contributes its main implementation directory. Tests, vendored third-party code, benchmarks, examples and fuzzers are excluded by path, and 130 generated files by name or by a generation marker in their first 4 kB. That exclusion is not cosmetic: before it, the highest-complexity function in `postgres` was a Snowball-generated stemmer and the deepest nesting in `protobuf` was its own compiler's output.

Project	Files	Code	Fns	med NLOC	p90	med	p90 CCN	max
sqlite	144	129,203	3,971	14	53	3	15	1,330
redis	214	140,575	5,815	11	43	3	12	326
nginx	403	179,735	3,236	27	91	5	20	190
curl	384	125,481	3,825	16	60	4	17	90
postgres	848	639,586	17,664	17	72	3	14	258
leveldb	89	11,197	857	5	22	1	5	33
rocksdb	559	162,453	9,516	5	30	1	6	163
protobuf	561	202,746	11,913	7	30	1	5	103
fmt	20	13,783	1,201	4	13	1	4	77
spdlog	107	20,742	1,833	4	13	1	4	77
json	47	19,160	853	5	29	1	10	273
ripgrep	141	33,992	2,183	6	25	1	6	42
tokio	293	45,432	3,942	5	18	1	4	25
serde	45	11,849	687	7	33	2	8	73
hyper	61	15,643	1,034	6	26	1	6	43
fd	22	3,891	189	7	32	1	8	37

Table 6: Sixteen public projects, generated files excluded. Commits are in `external.json`.

6.1 The indirection is visible before an agent runs

	Code lines	Fns per KLOC	med NLOC	med CCN	max CCN	xz per KLOC
C	1,214,580	28.4	11 to 27	3 to 5	90 to 1,330	4,217
C++	430,081	60.9	4 to 7	1	33 to 273	4,754
Rust	110,807	72.5	5 to 7	1 to 2	25 to 73	5,126

Table 7: The three languages over their public corpora. Medians and maxima are the range across that language’s projects.

C++ carries 60.9 functions per thousand code lines against C’s 28.4, and Rust 72.5. The median C++ and Rust function is 4 to 7 lines at cyclomatic complexity 1, against 11 to 27 lines at complexity 3 to 5 in C. Complexity 1 at the median means that function contains no branch at all: it is straight-line code that calls something else. Compressed density runs the same way, 4,217 bytes per thousand lines in C against 4,754 and 5,126.

That is this paper’s axis, in released code by other authors, without an agent and without a hypothesis. The path an agent walks is longer because there are two to three times as many functions per unit of behaviour, and the tokens it reads to reach the point of change are spread over more artifacts.

The maxima run the other way, and the reversal is sharp. The worst function in all of `tokio` is CCN 25, in `hyper` 43, in `ripgrep` 42; the worst in `sqlite` is 1,330 and in `postgres` 258. Rust carries the most indirection of the three and the least concentration of it. A language buys the absence of a tail with hops, and the two costs fall on different readers: a 601-line function at complexity 253 is what defeats a person, and a chain of seven four-line functions is what a person handles by not looking at it. Which one defeats an agent is the question Section 10 is for. None of this settles it. It establishes that the thing whose cost is in question is real, large, ordered, and not peculiar to the author.

6.2 Where the maximum sits

In the author’s thirteen projects every maximum above complexity 10 was an input parser. Over these sixteen it holds in thirteen. Nine parse an input: `sqlite3VdbeExec` interprets a bytecode stream, `rdbLoadObject` decodes a database file format, the `nginx` request-line parser handles HTTP,

`InterpretArgument` and `from_low_args` parse command lines, `parse_chrono_format` reads a format string in both `fmt` and `spdlog`, a `MessagePack` reader decodes a wire format, and `serde`'s `from_ast` parses attribute syntax. Four cross the same boundary outward: `ExplainNode` walks a plan tree to produce text, `curl_easy_strerror` maps internal codes to user-facing strings, `hyper` encodes headers, and `tokio`'s `worst` function is a `Display` implementation.

It fails three times, all the same kind. `leveldb`'s `DoCompactionWork` at CCN 33, `rocksdb`'s `crc32c_3way` at CCN 163 and `fd`'s `spawn_senders` at 37 are interior: a compaction driver, a hand-unrolled checksum kernel, and a thread launcher. Optimised or concurrent interiors accumulate branches for reasons unrelated to a program's edges.

The surviving claim is weaker than the author's own code suggested, and is stated at its measured strength: the highest-complexity function in a project is usually the one that parses its input, and the exceptions here are optimised kernels. Four of them cross it outward rather than inward, which is the same boundary. Twenty-four of the twenty-seven projects across both corpora that carry a tail at all. Reading that number as a verdict on a language or a style guide remains wrong, which is the use this paper objects to.

6.3 The same question in Java

System A serves 37 tables through 39 persistence classes and one data carrier, with no ORM. The conventional stack states the same behaviour across an entity, a repository, a service, a controller and a mapper. Thirteen public Java applications that persist through JPA were measured the same way, six of them data-centric systems comparable to System A in shape (`fineract`, `BroadleafCommerce`, `openmrs-core`, `syncope`, `shopizer`, `hawkbit`) and seven platform or sample applications where most of the code is not persistence at all.

	Entities	Persistence classes per entity	Layer classes per entity	Declared artifacts
<code>fineract</code>	255	2.16	5.4	n/a
<code>BroadleafCommerce</code>	159	1.42	3.3	188
<code>openmrs-core</code>	86	1.29	1.7	152
<code>syncope</code>	85	1.61	1.6	482
<code>shopizer</code>	82	2.44	4.3	73
<code>hawkbit</code>	21	1.14	3.6	n/a
median of the six		1.52	3.4	
System A	37 tables	1.08	1.08	8 deployed jars

Table 8: Public JPA applications against System A. Persistence classes are entity, repository and mapper; layer classes add service and controller. Declared artifacts are distinct Maven `artifactIds` across the project's POMs.

The persistence layer is the part that barely differs: 1.52 classes per entity against 1.08, a factor of 1.4. Whatever the ORM costs, it is not paid in classes at the point where rows become objects. The factor reaches 3.4 only when the service and controller layers are counted, and it reaches nothing like the whole-repository ratios, which range from 10 to 115 files per entity and measure how much of a system is not CRUD rather than what the stack costs.

Where the difference is not marginal is the dependency surface. System A deploys 8 jars totalling 3.3 MB, of which its own machinery is under 1 MB, and carries no `package.json`: its 46,390 lines of front end are plain JavaScript with no framework, no npm and no build step. The comparable systems declare 73 to 482 distinct Maven artifacts, and `thingsboard` adds 554 npm packages in its lockfile. Each is a version to track, a licence to clear and a vulnerability feed to watch, and none of it appears in a line count.

Two limits. Entity counts come from the `@Entity` annotation, so a system persisting through MyBatis registers as zero and drops out: `mall` at 519 files and `JeecgBoot` at 864, both large production systems, keep hand-written SQL in 76 and 80 `*Mapper.xml` files and use no ORM. The assumption that the ORM stack is the default does not survive that observation. And thirteen applications chosen by hand from public hosts are not a sample of industry practice; they are what could be found, measured, and re-measured by anyone with the script.

6.4 Feature rates

Table 9 gives the counts the pairs of Section 10 are parameterised by, so the C++ side is built at rates that occur rather than rates that favour the hypothesis. Counts are lexical, over comment-stripped source; a semantic count would need each project’s build configuration.

Project	Code	virtual	override	template	operator	inherits	smart ptr	lambda
leveldb	11,197	7.2	18.1	2.4	4.1	3.5	0.0	0.0
rocksdb	162,453	9.8	14.1	2.5	3.1	2.4	13.1	1.6
protobuf	202,746	1.5	6.1	10.0	3.1	1.8	2.3	2.2
fmt	13,783	0.1	0.3	67.5	7.1	1.9	0.5	1.7
spdlog	20,742	1.1	5.9	57.6	6.3	5.1	12.2	1.6
json	19,160	0.9	0.4	32.5	11.6	0.7	0.3	1.8
All	430,081	4.7	9.0	12.1	3.8	2.2	6.6	1.9

Table 9: C++ feature occurrences per 1,000 code lines, counted lexically.

The variance is larger than the mean and it is structured. `fmt` and `spdlog` carry 67.5 and 57.6 template declarations per thousand lines as header-only generic libraries; `leveldb` carries 2.4. Virtual dispatch runs the other way, 9.8 and 7.2 in `rocksdb` and `leveldb` against 0.1 in `fmt`. Library C++ and systems C++ use different halves of the language, so the mixed condition of Section 10 is built at systems rates, that being what an `ais` or a `System A` would become, and the paper reports the choice.

The corpus is six libraries and no applications. C++ applications are under-represented on public hosts, and their mix is likely different again; these rates are library and systems C++, not C++ in general.

7 What a representation costs to hold

Three costs are not source length: the information the source actually carries, the memory the program takes to run, and the prose that has to be written and kept true beside it.

7.1 Compressed size

Kolmogorov complexity is uncomputable, and compressed size is its standard proxy. Each set’s source is concatenated in a fixed order, comment-stripped, and compressed with `xz` at preset 9. Cross-file redundancy is counted deliberately: boilerplate repeated across files is what a compressor, and a reader who has already seen one instance of it, gets nearly free.

Same behaviour, one author	Lexical	Raw bytes	gzip	xz
bench C / Ada	0.81	0.38	0.63	0.66
bench C / Rust	1.42	0.97	1.17	1.15
bench C / Java	1.27	0.89	1.03	1.02
bench C / Python	1.74	1.30	1.51	1.43
mincdp C / Java	2.66	2.04	1.81	1.73
linearr C / Java	2.60	2.89	2.77	2.65

Table 10: Ratios of C to the other language on the same behaviour, by four measures. Comment-stripped source throughout.

The four measures disagree, and the disagreement has a direction. Compression removes most of the C’s excess where the behaviour is genuinely matched: on the bench the C/Java ratio falls from 1.27 lexical to 1.02 compressed, and on `mincdp` from 2.66 to 1.73. It removes none of `linearr`’s, because that C carries a scoring mode the Java lacks, which is why its ratio was already an upper bound. What compression takes away is repetition, and repetition is most of what explicit style adds.

Across the public corpora, where behaviour cannot be matched, the comparison is per line: C++ carries 4,754 compressed bytes per thousand code lines against C’s 4,217. A C++ line holds 13% more information than a C line, which is the same fact seen from the other side.

7.2 Memory

The five equivalent programs were run on identical input and their peak resident set recorded. Two points bracket it: the runtime’s floor, on a 400 and 350 element pair, and a data-dominated run on two lists of 2,000,000 ids, 17 MB each.

Language	Floor (750 ids)		2,000,000 ids each	
	Peak RSS	× C	Peak RSS	× C
C	1.7 MB	1.0	49.4 MB	1.0
Rust	2.2 MB	1.3	49.8 MB	1.01
Ada	5.0 MB	3.0	69.0 MB	1.4
Java	46.4 MB	27.9	128.5 MB	2.6
Python	10.0 MB	6.0	624.9 MB	12.7

Table 11: Peak resident memory on the intersection, same input, same machine. Wall time at the large input: C and Rust 0.16 s, Ada 0.23, Java 0.27, Python 4.44.

Rust costs 1% more than C at the large input and 0.16 s against 0.16 s, which is what a zero-cost abstraction means when it is measured rather than asserted. The runtime floor is the number that matters where the unit of deployment is a process: 46.4 MB against 1.7 MB is the same program.

7.3 Comments

If abstraction is a compression of the source, it is a lossy one: what runs cannot be recovered from the compressed form alone, and the missing part is supplied by an interface, a construction site, a convention, or a comment. Comments are the only one of those that nothing checks, and the only one that can silently stop being true.

For one behaviour by one author the prediction holds. The more abstract side of each pair carries more comment per line of code: 8.9 against 0.0 on the bench, 23.8 against 15.3 on `mincdp`, 41.6 against 26.0 on `linearr`, comment lines per hundred code lines. Over the author’s whole corpus it is 41.8 for the Java against 25.0 for the C.

Across projects by different authors the relation reverses: 38.2 for the five public C projects against 22.9 for the six C++. That comparison holds neither behaviour nor author fixed, and the within-language

spread swamps the between-language difference, from 4.4 in `nginx` to 50.6 in `postgres`. Comment density is a property of a project’s convention before it is a property of its language.

What a line count cannot see is whether a comment is needed. An API description and an explanation of what a template instantiation resolves to are both comment lines. Separating them is a reading task, not a counting one, and it is not attempted here.

8 Running an agent on the code

8.1 Protocol

The five bench implementations are the five conditions. Each run copies one into a fresh directory holding only the source file, a neutral `README.md` with build and run commands, `check.sh` (builds and runs both operations on the visible fixtures), and `fixtures/` (a 24-line store and two posting lists of nine and seven elements). The directory is a fresh git repository; trace, diff and verdict are written outside it.

Four tasks, one text each, identical for every language, naming behaviour and output but never a file, a function or a language feature: add a set-difference operation over the two lists; make the scan case-insensitive for ASCII letters; find and fix an injected defect in the intersection, one substitution per language that makes the merge advance the wrong side; and print the first and last common id after the intersection’s rows. The texts are in `pilot/tasks.py`.

Each task ends: keep everything else unchanged, verify with `check.sh`, do not ask questions.

The agent is Claude Code 2.1.241 headless, model `claude-sonnet-5`, default system prompt plus one sentence (work only in the current directory, do not ask questions), default tools, no MCP servers, no user settings or memory, at most 50 turns and \$4 a run. It is told nothing about the study, the other languages or the hidden tests. The 60 runs (4 tasks $\times$ 5 languages $\times$ 3 repetitions) ran in seeded random order, four at a time.

Hidden tests, never in the run directory, judge each result: the program is rebuilt from a clean tree with the `README`’s command and run on a 400-line store whose last line has no newline, and on five posting-list pairs (random with overlap, identical, disjoint, one empty, overlap only at the last element). Every printed row must carry the right count, Python printing two rows per operation and both having to be right, and the task’s own output must be right on every pair. All five originals pass the non-task checks; all five bugged copies fail the intersection.

Per run, from the stream-json trace: context tokens (the sum over API calls of input, cache-read and cache-creation tokens, what the model was shown cumulatively), output tokens, turns, tool calls by tool, files read, shell commands, edits, wall time, cost, hidden-test verdict, final diff.

8.2 Results

All 60 runs passed the hidden tests, and every run stopped of its own accord: no turn or budget cap was reached. Table 12 gives each language over the four tasks; the per-cell figures are in `pilot/runs`.

Language	Pass	Context median	Output median	Turns	Tool calls	Wall s	Cost \$
C	12/12	201,029	1,974	8	8	25	1.30
Ada	12/12	226,474	2,929	9	8	34	1.52
Rust	12/12	200,394	1,992	8	7	26	1.32
Java	12/12	200,200	1,887	8	7	27	1.32
Python	12/12	230,900	2,476	8	8	30	1.39

Table 12: Each language over the four tasks, twelve runs. Cost is the sum.

Per run the median context was 202,386 tokens (163,394 to 324,890), median output 2,078 (912 to 4,854), 6 to 12 turns, 5 to 11 tool calls. Every run read the source file first; the median run read three files (source, `README.md`, `check.sh`), ran the shell two to four times and made one or two edits. In every

T1 run the agent also updated `README.md` and `check.sh`. The 60 runs cost \$6.85 and 29 minutes: \$0.11 and 29 s a run.

Where the context goes: the first API call of every run shows the model 30,920 to 30,969 tokens (default system prompt, tool schemas, task), and each later call adds a median 926. The source file at o200k is 478 to 1,373 tokens; shown on every call it is at most 2.7% of a run’s context at the median (Ada 4.5%, Python 1.6%). Context is dominated by re-sending the accumulated conversation on every call: the median run pays 25,603 tokens a turn, with a two-fold spread across runs (17,221 to 34,287). It tracks turns only loosely. Regressed on turn count over the 60 runs, turns explain 27% of the variance, and a quadratic term adds three points more; six, seven and eight turn runs all land near 200k, and above eight the median rises about 20k a turn. What the regression does establish is where the money goes. The unit of cost is an exchange carrying the whole conversation so far, not a file: a task needing three more turns late in a session pays three times a context of 200k, while reading twice as much source costs a few thousand once.

Between languages: C, Rust and Java share a median of 200k to 201k; Ada 226k and Python 231k, 13 to 15% more. Within a cell, three repetitions spread by a median 34,652 tokens and up to 107,310, so the between-language differences are of that order. The rank changes with the task: C is cheapest on T1, second on T2 and T3, third on T4; Rust is cheapest on T3 and costliest on T1. Ada costs the most output (2,929 median against 1,887 to 2,476), which is verbosity on the writing side, its reading side being 4.5% of context. Python’s T2 is the costliest T2 (240k context, 4,027 output, 47 s): its scan has two paths and both had to change.

8.3 What the pilot does and does not show

With retrieval at its floor and tasks this small, representation does not show in the cost. The agent read the file once at 1 to 4% of its context and spent the rest re-sending the conversation, once per exchange. The five are separated only by turns taken and tokens written, and at three repetitions that separation sits inside the run-to-run spread. This is the null the design allows for, at the one scale where it was always likeliest.

Each repository is one file, so retrieval is one Read: the pilot measures reconstruction and modification with retrieval held at its floor, the condition the design calls “highly relevant context given”. Three repetitions bound the spread; they do not support a test between languages. One model was run. The bench is 25 to 133 lines, and nothing here scales.

9 Threats to validity

One author. Every set measured is written or shaped by the same person, whose C style is the one under study, and the `linearrr` twin shows a port inherits the original’s function shape. Nothing here separates “the C style” from “this author”; independent implementations of the equivalent sets are required for that. Section 6 answers it for the corpus measurements only.

Fifteen years. The genomics toolkits carry 2009 and 2011 in their file headers and no Java commit after their 2015 import; the C corpus is 2026, `bpnn` reaching back to 2015. Era, domain and language all differ between the groups, so their lower tail is not attributable to the language on this evidence. It does rule out the CCN 253 tail being a constant of the author.

Deliberately terse bench. `bench.c` packs 28 tokens per line and `Bench.java` is commented “deliberately terse”. The bench measures the author at his densest, and its LLM-token ratios do not transfer to ordinary density, as `mincdp` and `linearrr` show.

Python’s two paths. The Python bench prints an idiomatic and an explicit row per operation, so a task touching the scan must change both. That is a different task shape, and the occurrence-versus-line difference limits the fixtures.

Lexical feature counts. The C++ rates of Table 9 are regex counts over comment-stripped source, not semantic counts. A `template` in a comment-free string literal would be counted; a `template` instantiated without the keyword nearby would not.

Tokenizer proxy. o200k is not the model’s tokenizer.

Task authorship. The four tasks were written by the paper’s author after seeing the code. They are behaviour-specified and identical across languages, which removes the location hint but not the choice of which behaviours to ask for.

Model and version. One model at one date. Agent cost is a property of the pair (representation, model).

Equivalence. Established by the repositories’ own checks and this paper’s fixtures, on the domains they cover. The `mincdp` demo was not re-run.

10 The experiment this motivates

Section 6 compares C and C++ as other people write them, which holds neither behaviour nor author fixed. What follows holds both.

Porting `ais` to C++ would give one number with every feature confounded in it, so the unit of analysis is the indirection. Six pairs, each a small C program and the same behaviour in C++ using one construct, the sixth at a realistic mixture: direct call against virtual override, `void *` or duplication against template, named function against overloaded operator, explicit initialise and free against constructor and destructor, struct embedding against inheritance, and all together. Each pair passes one hidden suite, so equivalence is checked and not asserted. Rates come from Table 9, the mixed pair built at systems rather than header-only-library rates, so the C++ side is not a straw man and the choice is reported.

The pilot found cost to be a harness constant times turns with the source under 3% of context, so a large C++ file shows nothing a small one does not. What must be long is the resolution path: the construction site in one file, the override in another, the instantiation in a header. Seven artifacts can be seven short files, and the effect is predicted in turns. Four tasks per pair, at least two requiring a change whose effect is not visible at the site of the change. Six pairs at two conditions, four tasks and five repetitions is 240 runs, about \$26 and under two hours at four in parallel. The code lives with this paper beside `pilot/`, with its own build, kept out of `ais`.

What the runs yield is a cost per unresolved edge, which with the function density of Section 6 gives the cost of a codebase at a given size and density: a model with measured parameters, not a measurement of a port nobody wrote.

Two conditions follow it. A repository where retrieval is not one Read (the full `ais/c`, 10,597 code lines) crossed with ordinary tools against associative retrieval, which separates the two costs: if better retrieval closes the gap, the cost was retrieval. And a framework rung, one endpoint of the System A kind built three ways and judged at the HTTP boundary, since a framework hides control flow that is not in the repository in any form and so cannot be retrieved from it at all.

Outcomes are the three the pilot already records, per task type rather than pooled: tokens read and written, wall seconds, and turns to a passing hidden test with failures reported rather than filtered. Runtime cost is not among them, for the reason Section 7 gives. Mixed-effects regression on log context with task and model as random effects; five repetitions, since three resolved nothing below 15%.

H_0 : after task, model, size and retrieval, representation has no effect. H_1 : reconstruction cost, measured on the semantic graph, predicts agent cost beyond token count. H_2 : some abstractions raise it and some lower it, RAI and the standard containers being the candidates for lowering and framework control flow for raising it most.

10.1 Prediction, recorded first

No pair above existed on 24 August 2026, when this was written. The direction is recorded now so it cannot be fitted to the numbers later, with the results that would refute it.

1. **Local tasks**, the behaviour to change visible at the site of the change: no advantage to the explicit rungs, and a possible advantage to the abstract ones, since RAI removes a lifetime obligation the C carries on every path out of a function. Predicted loser: explicit C.

2. **Cross-module and debugging tasks**, where the answer depends on a relationship not visible at the site of the change: the explicit rungs cost less context and fail less often, the gap growing with plausible targets per call site rather than with source size.
3. **The mix**: an effect smaller than either, in whichever direction the task proportions decide.
4. **The framework condition** costs the most of any rung on cross-module tasks.

Table 13 gives the path an agent must walk to establish what one call does, for one behaviour: fetch a record and return it.

	Path from the call to the behaviour	Steps	No file
C, as written	caller, <code>ais_find</code> , the store read	3	0
C++, abstract	interface, construction site, override, template instantiation, constructor and destructor, operator overload, allocator	7	3
Java, as written	servlet, verb class, DAO returning the response body	3	0
Java, framework	controller, service, repository interface, its generated implementation, entity, mapper, DTO, with the transaction decided by a proxy	7	3

Table 13: Predicted path length for one behaviour, in two languages, at each end of the axis. *Steps* counts the artifacts an agent must resolve to establish what the call does. *No file* counts the steps whose behaviour has no textual origin anywhere in the repository. These are predicted values; measuring them is what the rungs are for.

The second column is what people argue about, and layered designs are defended on it fairly. The third has no answer for an agent, being behaviour that cannot be read at any price: a template instantiation, an implicit destructor, a generated repository implementation, a transaction boundary that exists only as an annotation. It is zero at both short paths and non-zero at both long ones, in two languages that share nothing else. The prediction is that agent cost tracks the third column rather than the second.

Three results refute it. The explicit rungs costing more context, or failing more often, on cross-module or debugging tasks. The gap closing when retrieval improves, which would mean the cost was retrieval and representation only its proxy. Or tokens written on the explicit rungs rising with the repetition explicitness requires, by enough to cancel the reading advantage, which would make the answer a property of the task mix. An effect below 15% of median context is not resolvable at three repetitions, so a null from too few will not be distinguishable from the pilot's.

11 Reproducibility

In this directory: `measure.py` (every source number; run it, and the commits it read are in its JSON), `tables.py` (the table bodies above, from the JSON), and `pilot/` with `mkfix.py` (the fixtures, seeded), `tasks.py` (the four task texts), `langs.py` (build, run and bug substitution per language), `setup.py` (templates, baseline and bug check), `run.py` (the harness), `hidden.py` (the hidden tests), `summarize.py`, and `runs/` with every run's `result.json`, `final.diff`, `hidden.log` and full `trace.jsonl`. The C and C++ pairs of Section 10, when written, go beside `pilot/` in this directory, with their own build and their own checker.

Toolchain: gcc 13.3.0, GNAT 13.3.0, rustc 1.75.0, OpenJDK 21.0.11, CPython 3.12.3, cloc 1.98, lizard 1.24.0, tiktoken 0.14.0, libclang 18.1.1, Claude Code 2.1.241, model `claude-sonnet-5`, Linux 6.8.

12 What is open

C, C++ and Rust are compared here as other people write them, over 1.76 million lines. What that cannot do is hold behaviour fixed: no two of those projects do the same job, so the ratios are per line rather

than per unit of work. The pairs of Section 10 close that gap for 240 runs and about \$26, and Section 10 records the predicted direction and the three results that would refute it beforehand.

What holds without them: the indirection whose cost is in question is real, large and ordered in released code, 28.4 functions per thousand lines in C against 60.9 in C++ and 72.5 in Rust, the last two at a median complexity of 1, and the maxima running the opposite way, from 1,330 in C down to 73 in Rust. Explicit style buys its extra length with redundancy, which compression removes and an abstraction's indirection does not. The long path is not required to deliver production behaviour, which one system of 39,621 lines and 37 tables shows by existing. And at one-file scale a coding agent's cost is a harness constant times its turns, with the representation under 3% of what it reads, so a one-file language comparison cannot answer this question at all.

References

- [1] V. Gavrilov. Repositories. <https://github.com/Anode1>.
- [2] V. Gavrilov. *ais: An extension of your associative memory*. <https://github.com/Anode1/ais,commit/84e3ad5>.
- [3] V. Gavrilov. *ais development style guide*. <https://github.com/Anode1/ais/blob/main/doc/dev/STYLE.md>.
- [4] V. Gavrilov. *Five languages, two core operations, one algorithm*. https://github.com/Anode1/ais/blob/main/tests/perf/LANG_COMPARISON.md.
- [5] V. Gavrilov. *mincdp: a minimal Chrome DevTools Protocol client in C and Java*. <https://github.com/Anode1/mincdp,commit/1bebbe3>.
- [6] V. Gavrilov. *linearr: least squares in C*. <https://github.com/Anode1/linearr,commit/0a79370>.
- [7] MISRA. *MISRA C:2012, Guidelines for the use of the C language in critical systems*. MIRA Ltd., 2013.
- [8] A. Danial. *cloc: Count Lines of Code*, version 1.98. <https://github.com/AlDanial/cloc>.
- [9] T. Yin. *lizard: a simple code complexity analyser*, version 1.24.0. <https://github.com/terryyin/lizard>.
- [10] OpenAI. *tiktoken*, version 0.14.0. <https://github.com/openai/tiktoken>.
- [11] Anthropic. *Claude Code*, version 2.1.241, headless mode. <https://docs.claude.com/en/docs/claude-code>.